НИТУ «МИСИС»

Кафедра инженерной кибернетики

ОТЧЕТ

ПО

ЛАБОРАТОРНОЙ РАБОТЕ №3

«Использование инструкции CPUID для

определения функциональных возможностей CPU»

учебная дисциплина «Параллельные вычисления»

Студент: Голубев И. М., БПМ-22-ПО-1

Преподаватель: Ватутин Э. И.

Оценка:

Дата защиты: 30.03.2026

2026 г.

Введение

Основная цель работы:

Познакомиться с базовыми принципами определения вычислительных возможностей и аппаратной конфигурации CPU.

Описание решаемой задачи

Задача:

Разработать программу для опроса процессора с использованием команды CPUID и вывода полученной информации в удобной для пользователя форме (оконное приложение, консольное приложение с выводом на экран или в файл).

Решено было реализовать консольное приложение с выводом информации в терминал.

Входные данные:

Дополнительных аргументов для консольного приложения не предусмотрено.

По сути, нужно реализовать Linux утилиту **cpuid** (см. Рис. 1)

```
 one-eyed-john@fedora  ~  cpuid
CPU 0:
   vendor_id = "GenuineIntel"
   version information (1/eax):
      processor type  = primary processor (0)
      family          = 0x6 (6)
      model           = 0x5 (5)
      stepping id     = 0x2 (2)
      extended family = 0x0 (0)
      extended model  = 0xa (10)
      (family synth)  = 0x6 (6)
      (model synth)   = 0xa5 (165)
   miscellaneous (1/ebx):
      process local APIC physical ID = 0x0 (0)
      maximum IDs for CPUs in pkg    = 0x10 (16)
      CLFLUSH line size              = 0x8 (8)
      brand index                    = 0x0 (0)
   brand id = 0x00 (0): unknown
   feature information (1/edx):
      x87 FPU on chip                        = true
      VME: virtual-8086 mode enhancement     = true
```

Рис. 1. Результат выполнения встроенной команды **cpuid**

Результат:

Максимально полная информация о процессоре.

Выбранные средства для разработки

Для выполнения лабораторной работы использовались следующие программные средства и технологии:

- Язык программирования: C++17
- Среда разработки: Zed
- Операционная система: Fedora Linux
- Процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
- Компилятор: g++
- Сборка программы: g++ -std=c++17 -O2 lab03.cpp -o build/cpuid_lab03

Листинг кода программы

Код доступен в удалённом Github репозитории по ссылке:

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab03/lab03.cpp

Ниже приведён полный листинг:

```
#include <cstdint>
#include <cstring>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>


struct CpuidRegs {
    uint32_t eax{}, ebx{}, ecx{}, edx{};
};


static inline CpuidRegs cpuid(uint32_t leaf, uint32_t subleaf = 0) {
    CpuidRegs r;

    #if defined(__i386__) && defined(__PIC__)
        asm volatile(
            "xchg{l} %%ebx, %1 \n\t"
            "cpuid                \n\t"
            "xchg{l} %%ebx, %1   \n\t"
            : "=a"(r.eax), "=&r"(r.ebx), "=c"(r.ecx), "=d"(r.edx)
            : "a"(leaf), "c"(subleaf)
            : "cc");
    #elif defined(__x86_64__) || defined(__i386__)
        asm volatile(
            "cpuid"
            : "=a"(r.eax), "=b"(r.ebx), "=c"(r.ecx), "=d"(r.edx)
            : "a"(leaf), "c"(subleaf)
            : "cc");
    #endif
```

```
    return r;
}


static inline uint64_t read_rflags() {
    uint64_t r;

    #if defined(__x86_64__)
        asm volatile("pushfq; popq %0" : "=r"(r) :: "cc");
    #else
        uint32_t e;
        asm volatile("pushfl; popl %0" : "=r"(e) :: "cc");
        r = e;
    #endif

    return r;
}


static inline void write_rflags(uint64_t r) {
    #if defined(__x86_64__)
        asm volatile("pushq %0; popfq" :: "r"(r) : "cc");
    #else
        uint32_t e = static_cast<uint32_t>(r);
        asm volatile("pushl %0; popfl" :: "r"(e) : "cc");
    #endif
}


static bool cpuid_supported() {
    constexpr uint64_t ID_BIT = 1ull << 21;

    uint64_t f1 = read_rflags();
    uint64_t f2 = f1 ^ ID_BIT;
    write_rflags(f2);

    uint64_t f3 = read_rflags();
    write_rflags(f1); // restore

    return ((f3 ^ f1) & ID_BIT) != 0;
}


static std::string vendor_string(const CpuidRegs& r0) {
    char v[13];

    std::memcpy(v + 0, &r0.ebx, 4);
    std::memcpy(v + 4, &r0.edx, 4);
    std::memcpy(v + 8, &r0.ecx, 4);
    v[12] = '\0';

    return std::string(v);
}


static std::string brand_string(uint32_t max_ext_leaf) {
    if (max_ext_leaf < 0x80000004u) {
        return {};
    }
    char b[49];

    std::memset(b, 0, sizeof(b));
```

```
    uint32_t* p = reinterpret_cast<uint32_t*>(b);

    for (uint32_t leaf = 0x80000002u; leaf <= 0x80000004u; ++leaf) {
        CpuidRegs r = cpuid(leaf, 0);
        *p++ = r.eax; *p++ = r.ebx; *p++ = r.ecx; *p++ = r.edx;
    }

    b[48] = '\0';
    return std::string(b);
}


static bool bit(uint32_t x, unsigned b) { return (x >> b) & 1u; }


static void print_hex_regs(const CpuidRegs& r) {
    auto old = std::cout.flags();
    std::cout << std::hex << std::setfill('0')
              << std::setw(8) << r.eax << ":"
              << std::setw(8) << r.ebx << ":"
              << std::setw(8) << r.ecx << ":"
              << std::setw(8) << r.edx;
    std::cout.flags(old);
}


struct CacheInfo {
    uint32_t level;
    uint32_t type;
    uint32_t line_size;
    uint32_t partitions;
    uint32_t ways;
    uint32_t sets;
    uint32_t size_bytes;
    bool fully_associative;
    bool inclusive;
    uint32_t threads_sharing;
};


static std::string cache_type_name(uint32_t t) {
    switch (t) {
        case 1:  return "Data";
        case 2:  return "Instruction";
        case 3:  return "Unified";
        default: return "Unknown";
    }
}


static std::vector<CacheInfo> query_caches_intel_leaf4() {
    std::vector<CacheInfo> caches;

    for (uint32_t i = 0; ; ++i) {
        CpuidRegs r = cpuid(4, i);
        uint32_t cache_type = r.eax & 0x1Fu;
        if (cache_type == 0) {
            break;
        }

        uint32_t level = (r.eax >> 5) & 0x7u;
        bool fully_assoc = bit(r.eax, 9);
```

```
        uint32_t threads_sharing = ((r.eax >> 14) & 0xFFFu) + 1u;

        uint32_t line_size = (r.ebx & 0xFFFu) + 1u;
        uint32_t partitions = ((r.ebx >> 12) & 0x3FFu) + 1u;
        uint32_t ways = ((r.ebx >> 22) & 0x3FFu) + 1u;
        uint32_t sets = r.ecx + 1u;
        bool inclusive = bit(r.edx, 1);

        uint64_t size = uint64_t(line_size) * partitions * ways * sets;

        caches.push_back(CacheInfo{
            level,
            cache_type,
            line_size,
            partitions,
            ways,
            sets,
            static_cast<uint32_t>(size),
            fully_assoc,
            inclusive,
            threads_sharing
        });
    }

    return caches;
}


static void print_size(uint64_t bytes) {
    if (bytes >= (1ull << 20) && (bytes % (1ull << 20) == 0))
        std::cout << (bytes >> 20) << " MiB";
    else if (bytes >= (1ull << 10) && (bytes % (1ull << 10) == 0))
        std::cout << (bytes >> 10) << " KiB";
    else
        std::cout << bytes << " B";
}


int main() {
    std::ios::sync_with_stdio(false);

    if (!cpuid_supported()) {
        std::cerr << "CPUID is not supported on this CPU." << "\n";
        return 1;
    }

    CpuidRegs r0 = cpuid(0, 0);
    uint32_t max_basic = r0.eax;
    std::string vendor = vendor_string(r0);

    std::cout << "Vendor: " << vendor << "\n";
    std::cout << "Max basic leaf: 0x" << std::hex << max_basic << std::dec << "\n";

    // Leaf 1: version + feature flags
    if (max_basic >= 1) {
        CpuidRegs r1 = cpuid(1, 0);

        uint32_t stepping = (r1.eax >> 0) & 0xF;
        uint32_t model = (r1.eax >> 4) & 0xF;
        uint32_t family = (r1.eax >> 8) & 0xF;
        uint32_t proc_type = (r1.eax >> 12) & 0x3;
        uint32_t ext_model = (r1.eax >> 16) & 0xF;
```

```
        uint32_t ext_family = (r1.eax >> 20) & 0xFF;

        std::cout << "\nCPUID(1): ";
        print_hex_regs(r1);
        std::cout << "\n";
        std::cout << "Version fields:\n";
        std::cout << "  Stepping: " << stepping << "\n";
        std::cout << "  Model: " << model << " (ext " << ext_model << ")\n";
        std::cout << "  Family: " << family << " (ext " << ext_family << ")\n";
        std::cout << "  Processor type: " << proc_type << "\n";

        // EBX fields: logical processors count, APIC ID
        uint32_t max_logical = (r1.ebx >> 16) & 0xFF;
        uint32_t apic_id = (r1.ebx >> 24) & 0xFF;
        std::cout << "Topology (from EBX):\n";
        std::cout << "  Max logical processors (legacy): " << max_logical << "\n";
        std::cout << "  Local APIC ID: " << apic_id << "\n";

        // Basic feature bits
        std::cout << "Features (EDX):"
                  << (bit(r1.edx, 0) ? " FPU" : "")
                  << (bit(r1.edx, 4) ? " TSC" : "")
                  << (bit(r1.edx, 23) ? " MMX" : "")
                  << (bit(r1.edx, 25) ? " SSE" : "")
                  << (bit(r1.edx, 26) ? " SSE2" : "")
                  << (bit(r1.edx, 28) ? " HTT" : "")
                  << "\n";

        std::cout << "Features (ECX):"
                  << (bit(r1.ecx, 0) ? " SSE3" : "")
                  << (bit(r1.ecx, 9) ? " SSSE3" : "")
                  << (bit(r1.ecx, 12) ? " FMA3" : "")
                  << (bit(r1.ecx, 19) ? " SSE4.1" : "")
                  << (bit(r1.ecx, 20) ? " SSE4.2" : "")
                  << (bit(r1.ecx, 28) ? " AVX" : "")
                  << "\n";
    }

    // Leaf 4: caches for Intel; iterate subleaf ECX until EAX==0
    if (max_basic >= 4) {
        std::cout << "\nCaches (CPUID(4, i)):\n";
        auto caches = query_caches_intel_leaf4();
        for (const auto& c : caches) {
            std::cout << "  L" << c.level << " " << cache_type_name(c.type)
                      << " cache: ";
            print_size(c.size_bytes);
            std::cout << "\n";
            std::cout << "    line_size=" << c.line_size
                      << " partitions=" << c.partitions
                      << " ways=" << c.ways
                      << " sets=" << c.sets << "\n";
            std::cout << "    fully_associative=" << (c.fully_associative ? "yes" :
"no")
                      << " inclusive=" << (c.inclusive ? "yes" : "no")
                      << " threads_sharing=" << c.threads_sharing
                      << "\n";
        }
        if (caches.empty()) std::cout << "  (no data)\n";
    } else {
        std::cout << "\nCaches: CPUID leaf 4 not supported by basic leaves.\n";
        std::cout << "Note: On AMD, cache info is typically via 0x8000001D instead
of leaf 4.\n";
```

```
    }

    // Leaf 7
    if (max_basic >= 7) {
        CpuidRegs r70 = cpuid(7, 0);
        std::cout << "\nCPUID(7,0): ";
        print_hex_regs(r70);
        std::cout << "\n";
        std::cout << "Extended features (EBX):"
                  << (bit(r70.ebx, 5) ? " AVX2" : "")
                  << (bit(r70.ebx, 11) ? " RTM" : "")
                  << (bit(r70.ebx, 29) ? " SHA" : "")
                  << (bit(r70.ebx, 16) ? " AVX512F" : "")
                  << "\n";
        std::cout << "Extended features (ECX):"
                  << (bit(r70.ecx, 8) ? " GFNI" : "")
                  << "\n";
        std::cout << "Extended features (EDX):"
                  << (bit(r70.edx, 24) ? " AMX-TILE/AMX-*" : "")
                  << "\n";

        if (r70.eax >= 1) {
            CpuidRegs r71 = cpuid(7, 1);
            std::cout << "CPUID(7,1): ";
            print_hex_regs(r71);
            std::cout << "\n";
            std::cout << "More (EDX):"
                      << (bit(r71.edx, 8) ? " AVX10" : "")
                      << (bit(r71.edx, 9) ? " AMX-CPLX" : "")
                      << "\n";
        }
    }

    // Leaf 0x16: base/max/bus frequency in MHz (low 16 bits)
    if (max_basic >= 0x16) {
        CpuidRegs r16 = cpuid(0x16, 0);
        uint32_t base_mhz = r16.eax & 0xFFFFu;
        uint32_t max_mhz  = r16.ebx & 0xFFFFu;
        uint32_t bus_mhz  = r16.ecx & 0xFFFFu;

        std::cout << "\nFrequencies (CPUID(0x16)):\n";
        if (base_mhz == 0 && max_mhz == 0 && bus_mhz == 0) {
            std::cout << "  (not reported)\n";
        } else {
            std::cout << "  Base: " << base_mhz << " MHz\n";
            std::cout << "  Max (boost): " << max_mhz << " MHz\n";
            std::cout << "  Bus: " << bus_mhz << " MHz\n";
        }
    }

    CpuidRegs rext0 = cpuid(0x80000000u, 0);
    uint32_t max_ext = rext0.eax;
    std::cout << "\nMax extended leaf: 0x" << std::hex << max_ext << std::dec <<
"\n";

    if (max_ext >= 0x80000001u) {
        CpuidRegs rext1 = cpuid(0x80000001u, 0);
        std::cout << "CPUID(0x80000001): ";
        print_hex_regs(rext1);
        std::cout << "\n";
        std::cout << "Ext features (ECX):"
                  << (bit(rext1.ecx, 6) ? " SSE4a" : "")
```

```
                << (bit(rext1.ecx, 16) ? " FMA4" : "")
                << "\n";
        std::cout << "Ext features (EDX):"
                << (bit(rext1.edx, 31) ? " 3DNow!" : "")
                << (bit(rext1.edx, 30) ? " Ext3DNow!" : "")
                << "\n";
    }

    std::string brand = brand_string(max_ext);
    if (!brand.empty()) {
        std::cout << "Brand string: " << brand << "\n";
    }

    return 0;
}
```

Результаты

Результат запуска и работы консольного приложения показан на рисунке 2.

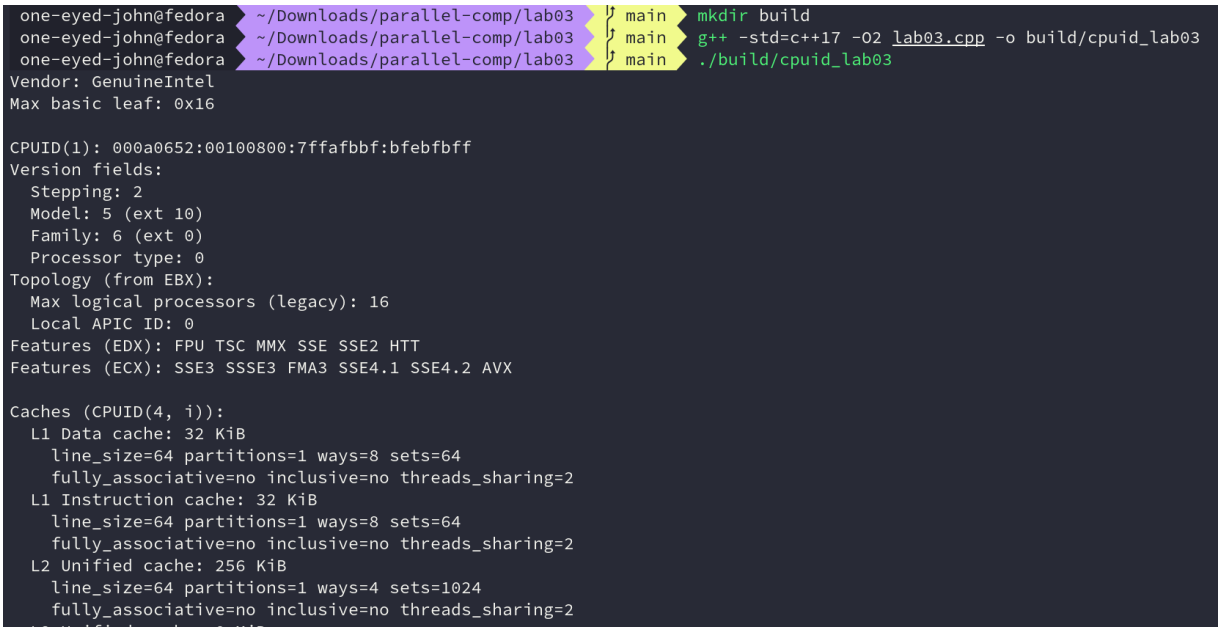

Рис. 2. Запуск и вывод приложения

Полный вывод программы приведён ниже:

Vendor: GenuineIntel
Max basic leaf: 0x16

CPUID(1): 000a0652:00100800:7ffafbbf:bfebfbff
Version fields:
  Stepping: 2
  Model: 5 (ext 10)
  Family: 6 (ext 0)
  Processor type: 0
Topology (from EBX):

```
  Max logical processors (legacy): 16
  Local APIC ID: 0
Features (EDX): FPU TSC MMX SSE SSE2 HTT
Features (ECX): SSE3 SSSE3 FMA3 SSE4.1 SSE4.2 AVX

Caches (CPUID(4, i)):
  L1 Data cache: 32 KiB
    line_size=64 partitions=1 ways=8 sets=64
    fully_associative=no inclusive=no threads_sharing=2
  L1 Instruction cache: 32 KiB
    line_size=64 partitions=1 ways=8 sets=64
    fully_associative=no inclusive=no threads_sharing=2
  L2 Unified cache: 256 KiB
    line_size=64 partitions=1 ways=4 sets=1024
    fully_associative=no inclusive=no threads_sharing=2
  L3 Unified cache: 8 MiB
    line_size=64 partitions=1 ways=16 sets=8192
    fully_associative=no inclusive=yes threads_sharing=16

CPUID(7,0): 00000000:029c67af:40000018:bc000600
Extended features (EBX): AVX2
Extended features (ECX):
Extended features (EDX):

Frequencies (CPUID(0x16)):
  Base: 2500 MHz
  Max (boost): 4500 MHz
  Bus: 100 MHz

Max extended leaf: 0x80000008
CPUID(0x80000001): 00000000:00000000:00000121:2c100800
Ext features (ECX):
Ext features (EDX):
Brand string: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
```

Выводы

Проведено знакомство с базовыми принципами определения вычислительных возможностей и аппаратной конфигурации CPU.

Результатом стала реализация стандартной утилиты cpuid.